Nina Nájera Marakovits - 231088

repo: https://github.com/Ninaswiftie09/ADA/tree/Lab_Rod_Cutting_Problem

Dynamic Programming: Rod Cutting Problem

This notebook shows different ways to solve the rod cutting problem. It includes a simple explanation, three implementations, small tests, and a time comparison.

1. Conceptual Analysis

Is dynamic programming a good choice for this problem?

Yes, dynamic programming is a good choice for this problem because the same smaller cases appear many times. For example, when solving a rod of length n, we also solve lengths like n - 1, n - 2, and so on. Instead of solving them again and again, we can save the results and reuse them.

What are the overlapping subproblems?

The overlapping subproblems are the best revenues for smaller rod lengths. For example, the value for length 3 may be needed many times while solving bigger lengths.

What is the optimal substructure?

The problem has optimal substructure because the best solution for a rod of length n can be built from the best solutions of smaller lengths. If we choose a first cut of size i, then we only need the best solution for the remaining part of length n - i.

In [46]:
```
# Price table used in the notebook
prices = [0, 1, 5, 8, 9, 10, 17, 17, 20, 24, 30]

# Example rod length
n = 8

print("Prices:", prices)
print("Rod length:", n)
```

```
Prices: [0, 1, 5, 8, 9, 10, 17, 17, 20, 24, 30]
Rod length: 8
```

2. Naive Recursive Solution

This solution tries every possible first cut. It follows the recurrence directly. It works, but it is slow because it solves the same subproblems many times.

In [47]:
```python
def rod_cut_recursive(prices, n):
    if n == 0:
        return 0

    best = float('-inf')

    for i in range(1, n + 1):
        best = max(best, prices[i] + rod_cut_recursive(prices, n - i))

    return best


for length in range(1, 9):
    print(f"Length {length}: {rod_cut_recursive(prices, length)}")
```

```
Length 1: 1
Length 2: 5
Length 3: 8
Length 4: 10
Length 5: 13
Length 6: 17
Length 7: 18
Length 8: 22
```

The recursive solution gives the correct answer for small values. For example, for length 4, the best revenue is 10. A good cut is 2 + 2, because price[2] + price[2] = 5 + 5 = 10.

3. Memoization

Memoization is a top-down dynamic programming method. It saves results that were already computed. This avoids repeated work and makes the solution faster.

In [48]:
```python
def rod_cut_memo(prices, n, memo=None):
    if memo is None:
        memo = {}

    if n == 0:
        return 0

    if n in memo:
        return memo[n]

    best = float('-inf')

    for i in range(1, n + 1):
        best = max(best, prices[i] + rod_cut_memo(prices, n - i, memo))

    memo[n] = best
    return best
```

```
for length in range(1, 9):
    print(f"Length {length}: {rod_cut_memo(prices, length)}")
```

```
Length 1: 1
Length 2: 5
Length 3: 8
Length 4: 10
Length 5: 13
Length 6: 17
Length 7: 18
Length 8: 22
```

This version is faster than the simple recursive one. The reason is that each subproblem is solved only once. After that, the saved value is reused.

4. Bottom-Up Dynamic Programming

The bottom-up method builds the answer from smaller lengths to bigger lengths. It uses a table and fills it step by step. This method is usually very efficient.

In [49]:
```
def rod_cut_bottom_up(prices, n):
    dp = [0] * (n + 1)

    for j in range(1, n + 1):
        best = float('-inf')
        for i in range(1, j + 1):
            best = max(best, prices[i] + dp[j - i])
        dp[j] = best

    return dp[n], dp


best_value, table = rod_cut_bottom_up(prices, 8)

print("Best revenue:", best_value)
print("DP table:", table)
```

```
Best revenue: 22
DP table: [0, 1, 5, 8, 10, 13, 17, 18, 22]
```

The table shows the best revenue for each rod length from 0 to n. This helps us see how the final answer is built step by step.

5. Checking That All Methods Give the Same Answer

Now we compare the three methods to make sure they return the same result.

In [50]:
```
for length in range(1, 9):
    r1 = rod_cut_recursive(prices, length)
```

```
    r2 = rod_cut_memo(prices, length)
    r3, _ = rod_cut_bottom_up(prices, length)

    print(f"Length {length}: recursive={r1}, memo={r2}, bottom_up={r3}")
```

```
Length 1: recursive=1, memo=1, bottom_up=1
Length 2: recursive=5, memo=5, bottom_up=5
Length 3: recursive=8, memo=8, bottom_up=8
Length 4: recursive=10, memo=10, bottom_up=10
Length 5: recursive=13, memo=13, bottom_up=13
Length 6: recursive=17, memo=17, bottom_up=17
Length 7: recursive=18, memo=18, bottom_up=18
Length 8: recursive=22, memo=22, bottom_up=22
```

All three methods return the same optimal revenue. This means the implementations are consistent. The main difference is performance.

6. Performance Comparison

In this part, we compare the running time of the three methods. The goal is to see how dynamic programming improves efficiency.

In [51]:
```
prices_large = [0] + [2 * i + (i % 3) for i in range(1, 101)]
print(prices_large[:15])
```

```
[0, 3, 6, 6, 9, 12, 12, 15, 18, 18, 21, 24, 24, 27, 30]
```

In [52]:
```
import time

def measure_time(func, *args):
    start = time.time()
    result = func(*args)
    end = time.time()
    return result, end - start
```

In [53]:
```
# Small value for recursive solution
n1 = 20

result_rec, time_rec = measure_time(rod_cut_recursive, prices_large[:n1+1], n1)
result_memo, time_memo = measure_time(rod_cut_memo, prices_large[:n1+1], n1)
result_bottom, time_bottom = measure_time(rod_cut_bottom_up, prices_large[:n1+1], n

print("n =", n1)
print("Recursive result:", result_rec, "| Time:", time_rec)
print("Memoized result:", result_memo, "| Time:", time_memo)
print("Bottom-up result:", result_bottom[0], "| Time:", time_bottom)
```

```
n = 20
Recursive result: 60 | Time: 0.26865601539611816
Memoized result: 60 | Time: 0.0
Bottom-up result: 60 | Time: 0.0
```

In [54]:
```
# Bigger value for dynamic programming methods
n2 = 100
```

```
result_memo_100, time_memo_100 = measure_time(rod_cut_memo, prices_large, n2)
result_bottom_100, time_bottom_100 = measure_time(rod_cut_bottom_up, prices_large,

print("n =", n2)
print("Memoized result:", result_memo_100, "| Time:", time_memo_100)
print("Bottom-up result:", result_bottom_100[0], "| Time:", time_bottom_100)
```

```
n = 100
Memoized result: 300 | Time: 0.0
Bottom-up result: 300 | Time: 0.0
```

7. Observations

The naive recursive solution is correct, but it is much slower. It repeats the same work many times.

The memoized version is faster because it saves previous results. The bottom-up version is also fast and builds the solution step by step.

Dynamic programming is a good approach for this problem because it reduces repeated work and gives the optimal answer efficiently.